

Karma Project

Game Systems Documentation

Engine	Unity 6000.0.62f1 (URP)
Genre	RPG - 6 Realms of Karma
Player Character	Sammy (3D, CharacterController)
MVP Chapter	Chapter 1: "Serna and the Empty Cup" (Hungry Ghost Realm)
Reference Games	It Takes Two, Little Nightmares
Scene	Assets/Scenes/Chapter1- Serna.unity

This document covers all game systems built for the Karma Project, including player mechanics, dialogue, karma economy, quest management, NPC behavior, UI, world objects, VFX, and editor tooling.

Table of Contents

Quick Start Setup

1. Architecture Overview

2. Player System

2.1 PlayerController | 2.2 State Machine | 2.3 Animation

3. Dialogue System

3.1 DialogueSO | 3.2 DialogueManager | 3.3 Actions & Conditions

4. Karma & Economy System

4.1 KarmaManager | 4.2 WalletManager | 4.3 VariableStore

5. Quest System

5.1 QuestSO | 5.2 QuestManager | 5.3 World Integration | 5.4 Fail-Soft Design

6. Interaction System

6.1 InteractableBase | 6.2 InteractionDetector | 6.3 World Objects

7. NPC System

7.1 NPCBase | 7.2 DialogueNPC | 7.3 GhostNPC

8. UI System

8.1 HUD | 8.2 Dialogue UI | 8.3 Quest UI | 8.4 Speech Bubbles

9. Environment & VFX

9.1 Food Rain | 9.2 Sparkle VFX | 9.3 Ghost Blocker

10. Editor Toolkit

10.1 Setup Tools | 10.2 Debug Windows | 10.3 Asset Creators

11. Design Patterns & Best Practices

12. How to Extend the Systems

Quick Start Setup

One-Time Setup (2 clicks + save)

1. Open your scene: Assets/Scenes/Chapter1- Serna.unity
2. **Karma > Setup Game Systems** -- auto-creates GameManagers with all 9 singletons (KarmaManager, WalletManager, DialogueManager, QuestManager, HUDManager, QuestLogUI, SparkleVFXManager, BackgroundMusicManager, GhostAtmosphere). Auto-assigns KarmaConfig, SparkleVFX prefab, BackgroundMusic clip, and all QuestSO assets.
3. **Karma > Build UI Canvases** -- builds HUDCanvas + DialogueCanvas + ChoiceButton prefab.
4. **Ctrl+S** to save the scene.

What's Automatic (No Manual Wiring)

- QuestLogUI auto-creates its tracker panel (top-right) and toast notifications (top-center) at runtime
- Quest actions/conditions auto-appear in the Dialogue Editor dropdown via reflection
- InteractionDetector auto-detects any InteractableBase in range
- QuickOutline auto-highlights targeted interactables
- NPCSpeechBubble auto-builds its UI if not pre-assigned
- BackgroundMusicManager auto-ducks when other audio plays

Creating Your First Quest

1. Right-click in Project > **Create > Karma > Quest** -- fill out Inspector
2. Place QuestTriggerZone (GoTo) / QuestItemPickup (Gather) in the scene
3. In Dialogue Editor: add StartQuestAction / AdvanceQuestAction / CompleteQuestAction
4. Re-run **Karma > Setup Game Systems** to pick up the new QuestSO

Debugging at Runtime

- **Karma > Quest Debug Console** (Play Mode) -- quest states, force-advance, event history
- **Karma > Variable Store** -- inspect/edit flags and counters live
- **Karma > Dialogue Editor** -- visual node graph editor

1. Architecture Overview

The Karma Project uses a **layered architecture** with clear separation between data definitions (ScriptableObjects), runtime state (singleton managers), event-driven UI, and world-space integration. All systems communicate via C# Action events -- no Update() polling for cross-system communication.

Core Layers

Layer	Components	Pattern
Data	DialogueSO, QuestSO, KarmaConfig, ItemSO, VariableSO	ScriptableObject (immutable at runtime)
Runtime State	QuestRuntimeState, SerializableDictionary	Plain C# [Serializable] classes
Managers	KarmaManager, WalletManager, DialogueManager, QuestManager, HUDManager, BackgroundMusicManager	Singleton + DontDestroyOnLoad
Events	OnKarmaChanged, OnDialogueStarted, OnQuestStarted, OnObjectiveUpdated, OnQuestCompleted, etc.	C# Action delegates
UI	DialogueUI, KarmaFlowerUI, CoinCounterUI, QuestLogUI, NPCSpeechBubble, KarmaPopupUI, CoinFlyUI	Event-driven, TrySubscribe pattern
World	QuestTriggerZone, QuestItemPickup, DialogueNPC, GhostNPC, PickupObject, PushableObject, ClimbSurface	MonoBehaviour + InteractableBase
Editor	GameSystemsSetup, DialogueEditorWindow, QuestDebugWindow, VariableStoreBrowser	EditorWindow + MenuItem

Singleton Managers

All managers live on a shared **GameManagers** GameObject (auto-created via Karma > Setup Game Systems). Each uses the standard singleton pattern with DontDestroyOnLoad and OnDestroy cleanup.

Manager	Events	Key API
KarmaManager	OnKarmaChanged, OnKarmaLevelUp	AddKarma(), CurrentLevel, GetNormalizedKarma()
WalletManager	OnCoinsChanged	AddCoins(), TotalCoins
DialogueManager	OnDialogueStarted, OnNodeChanged, OnChoiceMade, OnDialogueEnded	StartDialogue(), SelectChoice(), Advance(), EndDialogue()
QuestManager	OnQuestStarted, OnObjectiveUpdated, OnQuestCompleted, OnQuestFailed, OnObjectiveFailed	StartQuest(), AdvanceObjective(), CompleteQuest(), FailQuest(), FailObjective(), GetQuestsByTag()
HUDManager	(bridges events)	ShowHUD(), HideHUD(), ShowInteractionPrompt()
BackgroundMusicManager	(none)	ForceDuck(), PauseMusic(), ResumeMusic()

2. Player System

The player (Sammy) uses a **CharacterController** with a type-safe **state machine**, centralized input handling via New Input System, and a cached animator parameter system for zero-allocation animation control.

2.1 PlayerController

Main player manager owning CharacterController, input, animation, and state machine. Manages velocity, gravity, jump, stumble. Configurable speeds: moveSpeed, sprintSpeed, crouchSpeed, pushPullSpeed, climbSpeed.

2.2 State Machine

State	Behavior	Transitions
GroundedState	Idle/walk/sprint. Handles crouch/jump/interact.	-> Airborne, Crouch, Carry, PushPull, Climb
AirborneState	Jump/fall with coyote time & jump buffer.	-> Grounded (on land)
CrouchState	Reduced height/speed, stand-up clearance check.	-> Grounded (stand up)
CarryState	Hold physics objects, weight-affected throw. Smooth lerp pickup (0.2s ease-out).	-> Grounded (drop/throw)
PushPullState	Push/pull rigidbodies. AlignToNearestFace. Friction scaling. canPull enforcement.	-> Grounded (release), Airborne (edge)
ClimbState	Climb on ClimbSurface. climbHeight enforcement. Lateral boundary raycasts.	-> Airborne (jump off/drop)

2.3 Animation System

PlayerAnimationHandler uses a HashSet<int> validParams cache (built once in Awake from animator.parameters) for O(1) zero-allocation parameter validation. Hashed params: Speed, IsGrounded, VerticalVelocity, IsCrouching, IsCarrying, IsClimbing, IsPushing, IsSprinting. Triggers: Jump, DoubleJump, Land, Throw, Stumble.

3. Dialogue System

3.1 DialogueSO (Data Model)

Dialogue trees are defined as **ScriptableObject**s with nodes and choices. Each node can have conditions (gates), actions (side effects), per-node voice clips, and animation overrides. Choices support styled display (Empathetic/Selfish/Neutral).

Component	Key Fields	Purpose
DialogueSO	dialogueId, speakerName, nodes[]	Root asset. Create via Karma menu.
DialogueNode	nodeId, text, choices[], nextNodeId, conditions[], onShowActions[], voiceClip, nodeAnimation	Single dialogue beat. Conditions gate visibility. Actions fire on show.
DialogueChoice	text, nextNodeId, choiceStyle, conditions[], actions[]	Player response option. Actions fire on selection. ChoiceStyle sets color.

3.2 DialogueManager

Singleton that drives dialogue flow. Evaluates node conditions (skips if fail), executes onShowActions, tracks one-time rewards via runtime HashSet (resets each Play session). Properties: ActiveNPCSpeakerName, ActiveNPCTransform, CurrentNode, IsDialogueActive.

3.3 Extensible Actions & Conditions

The system uses **[SerializeReference]** for polymorphic serialization. New actions/conditions auto-appear in the editor dropdown via reflection (DialogueTypeCache). Zero core changes needed to extend.

Actions (IDialogueAction)	Conditions (IDialogueCondition)
ModifyKarmaAction - Add/subtract karma	KarmaLevelCondition - Check karma level
ModifyCoinsAction - Add/subtract coins	FlagCondition - Check VariableStore flag
SetFlagAction - Set VariableStore flag	CounterCondition - Check VariableStore counter
ModifyCounterAction - Modify counter	QuestStateCondition - Check quest state
StartQuestAction - Start a quest	QuestObjectiveCondition - Check objective progress
AdvanceQuestAction - Progress an objective	
CompleteQuestAction - Force-complete a quest	

How to Add a New Action (Zero Core Changes)

```
[Serializable]
public class ShowReflectionCardAction : IDialogueAction
{
    public string cardId;
    public string Label => $"Show Card: {cardId}";
    public void Execute() => ReflectionCardManager.Instance.ShowCard(cardId);
}
```

```
// Auto-appears in editor dropdown. No other files touched.
```

4. Karma & Economy System

4.1 KarmaManager

Singleton tracking the player's karma score and level. Uses **KarmaConfig** ScriptableObject for level thresholds, sprites, and audio clips. GetNormalizedKarma() returns 0.0-1.0 for NPC behavior evaluation.

Karma Flow: Player choice -> ModifyKarmaAction -> KarmaManager.AddKarma() -> OnKarmaChanged -> KarmaFlowerUI (animated progress bar with green flash) + KarmaPopupUI (fly-to-target animation). Level thresholds trigger OnKarmaLevelUp -> petal bloom + LevelUpFlash overlay.

4.2 WalletManager

Singleton tracking coin balance. OnCoinsChanged(total, delta) fires for UI updates. CoinFlyUI spawns 3 gold circles that stagger-fly to the CoinCounter. CoinCounterUI shows delta popup (+green, -red) with audio feedback.

4.3 VariableStore

ScriptableObject singleton (Resources.Load) storing game state: **flags** (bool), **counters** (int), **relationships** (int). Used by dialogue conditions, quest rewards, and fail-soft fallbacks. Parallel list serialization pattern (Unity cannot serialize Dictionary directly).

5. Quest System

A full quest lifecycle system inspired by Witcher 3 (facts database), Little Nightmares (silent/environmental beats), and Zelda (modular objectives). Supports quest chaining, semantic tags, objective visibility tiers, and a compassionate fail-soft design.

5.1 QuestSO (Quest Definition)

Immutable quest definitions created as ScriptableObjects via **Create > Karma > Quest**.

Field	Type	Purpose
questId	string	Unique identifier for save/load and cross-references
displayName	string	Name shown in quest log
description	string	Detailed description for quest details view
questType	QuestType enum	Main, Side, or Bounty (repeatable/challenge)
objectives[]	QuestObjective[]	Ordered list. First incomplete = active objective.
rewards	QuestRewards	karma, coins, items, flags, questsToUnlock
prerequisites[]	string[]	Quest IDs that must be Done before Available
autoStart	bool	Auto-start when all prerequisites met
isRepeatable	bool	Can be replayed after completion
tags[]	string[]	Semantic tags (e.g., 'realm_hungry_ghost', 'emotional')

Quest State Machine (FSM)

State	Description	Transitions To
Locked	Prerequisites not met	-> Available (when prerequisites done)
Available	Prerequisites met, not yet accepted	-> Active (StartQuest)
Active	Player is working on objectives	-> Completed (all objectives done)
Completed	All objectives done, awaiting rewards	-> Done (CompleteQuest)
Done	Rewards collected, quest fully resolved	(terminal, or -> Active if repeatable)
Failed	Quest failed (timed/failable quests)	(terminal)

Objective Configuration

Field	Type	Purpose
objectiveId	string	Unique ID within quest (e.g., 'talk_serna')
description	string	UI display text (e.g., 'Talk to Serna')
type	ObjectiveType	Talk, Gather, GoTo, Activate, Kill, Minigame, Custom

Field	Type	Purpose
targetId	string	NPC id, item id, location tag, minigame id
requiredCount	int (min 1)	How many times to complete (e.g., 'Collect 3 apples')
isOptional	bool	If true, not required for quest completion
visibility	ObjectiveVisibility	Hidden / SoftHint / JournalVisible / MapMarkerVisible
canFail	bool	If true, objective can fail (timed challenges, minigames)
retryAllowed	bool	If canFail: allow retry after failure (resets progress)
fallbackDialogueId	string	Dialogue to trigger on failure (compassionate fallback)

Objective Visibility (Little Nightmares-style)

Visibility	UI Behavior	Use Case
Hidden	No UI, no journal entry	Silent/environmental beats (player explores organically)
SoftHint	Subtle text only, no progress counter	NPC gives vague direction, no explicit tracking
JournalVisible	Shows in quest log, progress counter	Standard quest objectives
MapMarkerVisible	Full map marker + journal entry	Key objectives with waypoints

5.2 QuestManager

Singleton controller that owns the quest registry (Dictionary<string, QuestSO>) and runtime state (Dictionary<string, QuestRuntimeState>). All quest operations go through this manager.

Method	Behavior
StartQuest(questId)	Transitions Locked/Available -> Active. Initializes objective progress.
AdvanceObjective(questId, objId, amount)	Increments progress. Auto-completes quest when all required objectives met.
CompleteQuest(questId)	Awards rewards (karma, coins, flags) and transitions to Done. Unlocks follow-up quests.
FailQuest(questId)	Transitions Active -> Failed.
FailObjective(questId, objId)	Fail-soft cascade (see below). Smart handling based on objective config.
GetQuestState(questId)	Returns current QuestState enum value.
GetObjectiveProgress(questId, objId)	Returns (current, required) tuple.
GetActiveQuests()	Returns list of all Active QuestRuntimeState.
GetQuestsByTag(tag)	Returns all QuestSO definitions matching a semantic tag.
ResetAllQuests()	Clears all progress. Re-initializes from definitions.

Quest Flow Diagram

1. **Define:** Create QuestSO assets (Create > Karma > Quest)
2. **Register:** QuestManager loads all QuestSO[] on Awake, builds registry
3. **Initialize:** On Start, sets Available for quests with met prerequisites
4. **Start:** Dialogue StartQuestAction or autoStart -> QuestManager.StartQuest()
5. **Progress:** World objects (QuestTriggerZone, QuestItemPickup) or dialogue (AdvanceQuestAction) -> AdvanceObjective()
6. **Events:** OnObjectiveUpdated -> QuestLogUI tracker + toast notifications
7. **Complete:** All required objectives met -> auto-CompleteQuest() -> awards rewards
8. **Chain:** rewards.questsToUnlock -> UnlockFollowUpQuests -> next quest Available/auto-started

5.3 World Integration

Component	Objective Type	How It Works
QuestTriggerZone	GoTo	Collider trigger. Player enters zone -> AdvanceObjective(). singleUse flag. requireQuestActive check.
QuestItemPickup	Gather	InteractableBase extension. Player presses E -> AdvanceObjective(). Awards item karma/coins. Deactivates after pickup.
StartQuestAction	Talk (start)	IDialogueAction. Fires from dialogue node/choice -> QuestManager.StartQuest().
AdvanceQuestAction	Talk (progress)	IDialogueAction. Fires from dialogue -> QuestManager.AdvanceObjective().
CompleteQuestAction	Talk (finish)	IDialogueAction. Fires from dialogue -> QuestManager.CompleteQuest().

5.4 Fail-Soft Design (Compassionate Fallbacks)

The fail-soft system ensures players are never stuck. When `FailObjective()` is called, a multi-level cascade determines the response:

Configuration	Behavior
canFail = false	Ignored. Objective stays incomplete -- player must eventually succeed.
canFail + retryAllowed	Progress resets to 0. OnObjectiveFailed(retry=true). UI shows 'Try Again' toast. Player can retry immediately.
canFail + !retryAllowed + isOptional	Objective skipped. Quest continues without it. If remaining objectives complete -> auto-complete quest.
canFail + !retryAllowed + required + fallbackDialogId set	Sets VariableStore flag '{questId}_{objId}_failed' = true. NPCs can react with compassionate fallback dialogue.
canFail + !retryAllowed + required + no fallback	Quest fails entirely. OnQuestFailed fires.

6. Interaction System

A trigger-based, angle-weighted interaction system. The player's `InteractionDetector` uses a `SphereCollider` trigger to find nearby `InteractableBase` objects, scores them by dot product (facing direction), and shows the best target with a `QuickOutline` highlight.

Component	Purpose
<code>IInteractable</code>	Interface: <code>InteractionPrompt</code> , <code>CanInteract()</code> , <code>Interact()</code> .
<code>InteractableBase</code>	Abstract base. Default <code>QuickOutline</code> toggle on targeted/untargeted. Lazy-cached outline lookup via <code>outlineLookedUp</code> flag.
<code>InteractionDetector</code>	On <code>PlayerController</code> . Trigger-based detection. <code>MinDotThreshold=0.3f</code> (~72 deg). Allocation-free null cleanup. Events: <code>OnPromptChanged</code> , <code>OnPromptHidden</code> .
<code>DialogueNPC</code>	Extends <code>InteractableBase</code> . Starts dialogue on interact.
<code>QuestItemPickup</code>	Extends <code>InteractableBase</code> . Advances quest objective on interact.
<code>PickupObject</code>	Carriable items. Smooth lerp pickup. Weight-affected throw.
<code>PushableObject</code>	Push/pull physics. Friction scaling. <code>canPull</code> enforcement.
<code>ClimbSurface</code>	Climbable surfaces. <code>climbHeight</code> enforcement.

7. NPC System

Component	Base Class	Behavior
NPCBase	(abstract MonoBehaviour)	Detection radius, FacePlayer(), audio helpers, proximity events.
DialogueNPC	InteractableBase	QuickOutline pulsing, SernaAnimCycler, audio fade. Per-node voiceClip + nodeAnimation playback. TryCrossFade() checks Animator.HasState(). Default animation fields: defaultIdleClips[], defaultTalkClips[], defaultAnimChangeInterval.
GhostNPC	NPCBase	NavMesh roaming with 3 states: Roaming/Paused/Reacting. Karma evaluator for behavior. Greet/Scream reactions.
GhostFloatEffect	MonoBehaviour	Visual-only sine-wave floating/bobbing. Independent of GhostNPC movement.

8. UI System

8.1 HUD Layer

HUDCanvas (Screen Space Overlay, Sort Order 5) contains: KarmaFlower (top-left, animated progress bar with green flash), CoinCounter (below karma, delta popups), KarmaPopup (fly-to-target animation), CoinFlyUI (3 coins stagger-fly), InteractionPrompt (bottom-center), FPSCounter (top-right), ResetButton (bottom-right, resets karma/coins/VariableStore/quests/dialogue rewards + reloads scene).

8.2 Dialogue UI

DialogueCanvas (Sort Order 10). DialoguePanel: 700px fixed-width centered, orange border + cream background. Three modes: **NPC-no-choices** (panel hidden, speech bubble only), **NPC-with-choices** (ghost panel -- border/inner hidden, only floating ChoiceButtons visible), **Player speaking** (full panel with 'Sammy' badge). Typewriter effect. Z/X/C for choices, Enter/E/Space to advance.

8.3 Quest UI (QuestLogUI)

Auto-creates UI components if not pre-assigned in Inspector:

- **HUD Tracker**: Top-right panel showing active quest name + current objective. Skips Hidden objectives, shows SoftHint as text-only (no progress counter), JournalVisible with progress (e.g., '2/3').
- **Toast Notifications**: Top-center popup for quest events. 3-second display + 0.5-second fade-out via CanvasGroup alpha coroutine.

Toast messages: 'New Quest:', 'Objective Complete:', 'Quest Complete:', 'Quest Failed:', 'Try Again:'

8.4 NPC Speech Bubble

World-space canvas above NPCs. Dynamic height from renderer bounds. Brown name badge + speech text. Auto-billboards to camera. Typewriter effect (35 chars/sec) with IsTypewriting/SkipTypewriter() API. 'Press Enter >>' prompt after typewriter finishes. Fade in/out animation.

9. Environment & VFX

Component	Purpose
FoodRainManager	"Cloudy with a Chance of Meatballs" food rain. Object-pooled 3D food falling from sky. Auto-loads prefabs from Assets/Prefab/Environment/Food. Auto-detects tiny models + applies scale. Auto-assigns URP fallback material. StartRain()/StopRain() API.
FallingFood	Per-food behavior: fall, tumble, shrink near ground, deactivate. Managed by FoodRainManager pool.
SparkleVFX	Code-configured ParticleSystem burst. 25 faded yellow orbs, 0.8s. Auto-deactivates for pooling. URP-safe material loading.
SparkleVFXManager	Singleton pool/spawner (5 instances). PlayAt(), PlayAtTransform(), PlayAtPlayer(). Auto-expands.
GhostBlocker	Invisible wall blocking ghosts (NavMesh obstacle).
QuestTriggerZone	Trigger zone advancing quest objectives on player enter.
TriggerZone	General-purpose trigger zone with UnityEvents.
Lever / PressurePlate	Environmental puzzle elements with trigger events.

10. Editor Toolkit

All editor tools are accessible under the **Karma** menu in Unity.

10.1 Setup & Validation Tools

Menu Item	What It Does
Setup Game Systems	Auto-creates GameManagers with all singletons (KarmaManager, DialogueManager, QuestManager, WalletManager, HUDManager, SparkleVFXManager, BackgroundMusicManager, QuestLogUI). Auto-assigns QuestSO assets found via AssetDatabase.
Build UI Canvases	Programmatically creates HUDCanvas + DialogueCanvas + ChoiceButton prefab matching Figma mockup layout.
Quick Setup Checklist	Prints full setup guide to console.
Validate Player Setup	Checks player components, tag, animator.
Rebuild Player Animator	Rebuilds PlayerAnimatorController with all states/transitions.
Setup Food Rain	Creates FoodRainManager, auto-loads food prefabs.
Fix Food FBX Materials	Diagnoses and fixes FBX material import for invisible models.
Sync NPC Animator States	Auto-adds missing animation states to NPC AnimatorControllers.

10.2 Debug & Inspection Windows

Window	Features
Dialogue Editor (Karma > Dialogue Editor)	3-panel visual editor: node list with arrows, detail panel, embedded preview. Color-coded node cards.
Variable Store (Karma > Variable Store)	Inspect/edit flags, counters, relationships. Add/remove/search in play mode.
Quest Debug Console (Karma > Quest Debug Console)	Play-mode only. Tab-filtered quest list (All/Active/Locked/Done+Failed). Search by ID, name, tags. Per-quest: color-coded state badge, objectives with progress bars, +1/Max buttons. Force Start/Complete/Fail. Progression Blocker Finder (shows missing prerequisites). Event History log (last 20 events). VariableStore Quick View (flags toggle, counters +/-1).

10.3 Asset Creators

Menu Item	Creates
Create Karma Config	KarmaConfig ScriptableObject asset
Create Variable Store	VariableStore asset in Resources/
Create Serna Intro Dialogue	Sample Serna_Intro.asset with extensible actions

Menu Item	Creates
Create Serna Return Dialogue	Serna_Return.asset demonstrating conditions
Create Sparkle VFX Prefab	SparkleVFX prefab at Assets/Prefab/VFX/

11. Design Patterns & Best Practices

Pattern	Where Used	Why
State Machine	PlayerStateMachine (6 states), GhostNPC (3 states), QuestState FSM (6 states)	Clean separation of behavior. Type-safe transitions. Easy to add new states.
Singleton + DontDestroyOnLoad	KarmaManager, DialogueManager, WalletManager, QuestManager, HUDManager, BackgroundMusicManager	Global access. Persist across scene loads. OnDestroy cleanup.
ScriptableObject Data	DialogueSO, QuestSO, KarmaConfig, VariableStore	Inspector-editable. Asset-based. Immutable at runtime (definitions).
Immutable Def + Mutable State	QuestSO + QuestRuntimeState	SO holds definition (shared). Plain C# class holds runtime state (serializable for save/load).
Event-Driven	All managers fire C# Action events. UI subscribes. No Update() polling.	Decoupled systems. Zero per-frame cost when nothing changes.
Interface Extensibility	IDialogueAction, IDialogueCondition + [SerializeReference]	New types auto-discovered via reflection. Zero core changes.
TrySubscribe Pattern	All UI + camera scripts	Handles initialization order. OnEnable + Start fallback. Singleton may be null during OnEnable.
Lazy Caching	InteractableBase (outlineLookedUp), PlayerAnimationHandler (validParams)	Avoid repeated expensive lookups. O(1) access after first call.

Performance Optimizations

- PlayerAnimationHandler: HashSet validParams replaces try-catch on every SetBool/SetFloat (~10+/frame)
- InteractionDetector: Manual backward for-loop replaces RemoveAll(lambda) -- zero per-frame delegate allocation
- InteractableBase: Lazy QuickOutline lookup with outlineLookedUp flag avoids repeated GetComponentInChildren
- DialogueManager: Runtime HashSet for one-time rewards instead of ScriptableObject persistence
- BackgroundMusicManager: Coroutine-based audio scan every 0.15s with 2s cache refresh (not per-frame)
- All Debug.Log calls in InteractionDetector wrapped in #if UNITY_EDITOR

12. How to Extend the Systems

Add a New Dialogue Action

Create a [Serializable] class implementing IDialogueAction. It auto-appears in the Dialogue Editor dropdown via reflection. No changes to DialogueManager needed.

Add a New Dialogue Condition

Same pattern as actions -- implement IDialogueCondition with Label + Evaluate(). Auto-discovered by DialogueTypeCache.

Create a New Quest

1. Right-click in Project > Create > Karma > Quest
2. Fill in questId, displayName, objectives, rewards
3. Set prerequisites (quest IDs that must be Done first)
4. Add to QuestManager's questDefinitions array (or re-run Setup Game Systems)
5. Wire up world objects: QuestTriggerZone for GoTo, QuestItemPickup for Gather
6. Wire up dialogue: StartQuestAction, AdvanceQuestAction, CompleteQuestAction
7. Use QuestStateCondition / QuestObjectiveCondition to gate dialogue based on progress

Create a New Interactable

Extend InteractableBase and override CanInteract() + Interact(). Add a Collider (trigger or not). InteractionDetector auto-detects it. QuickOutline auto-highlights if the component is present.

Add a New Player State

Create a new class extending PlayerState. Register it in PlayerStateMachine. Add transition logic in existing states (e.g., GroundedState checks for new interaction type).

Add a New NPC Type

For dialogue NPCs: extend InteractableBase (like DialogueNPC). For AI NPCs: extend NPCBase (like GhostNPC). Both patterns support audio, animation, and proximity events.